PROGRAMMING IN JAVA

ASSIGNMENT 2 — CO2

STRINGS, COLLECTIONS, GENERICS AND COMPARATORS

Student Details	**To be completed**
Name	J.SUBHAASHREE
Register Number	192424008
Course / Section	AI&DS
Faculty	Ravi Pujitha
Submission Date	5.11.26
GitHub Repository Link	https://github.com/Subhaashree06/JAVA-Assignments

Q1 — BUILDING TEXT UNDER LOAD: STRING, STRINGBUILDER OR STRINGBUFFER?

1.1 Prediction and Explanation

String is immutable. Every concatenation creates a new String value, so repeated concatenation inside a loop repeatedly copies the characters already built. For n fields of similar length, the total copied characters can grow proportionally to $1 + 2 + \ldots + n$, giving $O(n^2)$ character-copying work.

StringBuilder is mutable and appends into a resizable character buffer. Its append operation is amortized O(1), so building a ticket with n fields is O(n) overall, excluding the cost of copying the final result. StringBuffer provides similar mutable behaviour but synchronizes its methods, adding overhead when only one thread is building a ticket.

Approach	Prediction	Typical use
String + in loop	Most allocations and copying as fields increase; $O(n^2)$ cumulative copying	Very small fixed concatenations
StringBuilder	Fastest single-threaded mutable builder; O(n) total append work	Build one ticket per request/thread
StringBuffer	Similar algorithmic complexity to StringBuilder, with synchronization overhead	Shared mutable buffer only when synchronized access is truly required

1.2 Three Workloads

Workload	Design	Purpose
A — Short ticket	1 ticket with 6 fields	Shows small-workload differences may be negligible.
B — High volume	10,000 tickets, 50 fields each	Exposes allocation/copying cost under load.
C — Long ticket	1,000 tickets, 500 fields each	Tests scaling when each ticket contains many fields.

1.3 Implementation

The following program implements all three approaches, repeats each workload, averages System.nanoTime() measurements, and estimates the number of intermediate concatenation results. Memory readings are approximate JVM heap observations and are affected by garbage collection.

```
import java.util.*;
```

Q1.4 Analysis and Design Decision

Expected trend: for six fields, differences may be too small to matter and measurement noise can dominate. For 10,000 × 50 and 1,000 × 500 workloads, StringBuilder should usually outperform repeated + because it avoids repeatedly rebuilding the accumulated prefix. StringBuffer is expected to be slightly slower than StringBuilder in a single-threaded benchmark due to synchronization.

Recommendation: use StringBuilder to build each individual ticket because each request/thread can own its own builder. Use a thread-safe logging mechanism for a shared booking log, such as a synchronized logger or a concurrent logging framework. Do not share one StringBuilder among booking threads. StringBuffer is an option for a genuinely shared mutable character buffer, but a dedicated logging API is generally safer.

Q1.5 Alternative: String.join, StringJoiner and String.format

String.join(delimiter, fields) and StringJoiner are suitable when fields are already available as strings and a delimiter is required. They avoid writing an explicit append loop and provide readable code. Their total work is generally O(total output characters), with storage for the final result and temporary bookkeeping. String.format is convenient for formatted templates but incurs formatting/parsing overhead and is usually not the fastest choice for high-volume simple concatenation.

```
String ticket = String.join("|", fields);
StringJoiner joiner = new StringJoiner("|");
for (String field : fields) joiner.add(field);
String ticket2 = joiner.toString();
String ticket3 = String.format("PNR:%s Passenger:%s Fare:%s", pnr, name, fare);
```

Method	Time complexity	Extra space
Repeated + in loop	$O(n^2)$ copied characters for n growing prefixes	Many intermediate String objects; cumulative allocation can be large
StringBuilder	O(L), where L is final output length	One growable buffer plus final String
StringBuffer	O(L) plus synchronization overhead	One synchronized growable buffer plus final String
String.join / StringJoiner	O(L)	Final result plus temporary references/builder state
String.format	At least O(L), with format-processing overhead	Formatter and output buffer allocations

Q2 WHEN IS ARRAYLIST AN APPROPRIATE ENGINEERING CHOICE?2.1 Experiment Design

Compare ArrayList<Song>, LinkedList<Song> and ArrayDeque<Song> at three sizes: 1,000, 10,000 and 100,000 songs. Measure add-at-end, insert/remove at front and middle, get(k), and full iteration. Use the same data and repeat operations several times after warm-up.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	Amortized O(1)	O(1) with tail reference	Amortized O(1) at either end
Insert at front	O(n) shifting	O(1) at head	O(1) addFirst
Insert in middle	O(n) shifting	O(n) to locate + O(1) link	Not supported by index; awkward
Get k-th	O(1)	O(n) traversal	No indexed access
Remove front	O(n) shifting	O(1)	O(1) removeFirst
Iterate all	O(n), cache-friendly	O(n), node traversal	O(n)
Space per element	Reference slot in backing array	Node with item + prev/next + references	Reference slot in circular array

2.2 Engineering Recommendation

ArrayList is a reasonable choice when users frequently jump to a song by index, append songs, and iterate through the playlist, while front/middle insertions are relatively infrequent. Stop recommending ArrayList when the dominant workload repeatedly inserts/removes at the front or in the middle and shifts a large number of elements.

For the changed Up Next queue, ArrayDeque is more appropriate because additions at both ends and removals from the front are amortized O(1). ArrayDeque does not support random indexed access, so it is not a replacement for the main indexed playlist.

2.3 Plain Array Alternative

A manually managed array can provide O(1) indexed access and O(1) append while capacity remains. Inserting/removing at the front or middle requires shifting O(n) elements. Resizing requires allocating a larger array and copying existing references, usually amortized O(1) per append when capacity grows geometrically.

2.4 Benchmark Program

```
import java.util.*;

public class ListBenchmark {
    static long time(Runnable task) {
        long s=System.nanoTime(); task.run(); return System.nanoTime()-s;
    }
    public static void main(String[] args) {
        for (int n : new int[]{1000,10000,100000}) {
            ArrayList<Integer> a=new ArrayList<>();
            LinkedList<Integer> l=new LinkedList<>();
            ArrayDeque<Integer> d=new ArrayDeque<>();
            for(int i=0;i<n;i++){a.add(i);l.add(i);d.addLast(i);}
            long aGet=time(()->{long x=0;for(int i=0;i<n;i++)x+=a.get(i);});
            long lIter=time(()->{long x=0;for(int v:l)x+=v;});
            long aIter=time(()->{long x=0;for(int v:a)x+=v;});
            long dIter=time(()->{long x=0;for(int v:d)x+=v;});
                System.out.printf("n=%d ArrayList get-all %.3f ms, iteration %.3f ms; LinkedList
iteration %.3f ms; ArrayDeque iteration %.3f ms%n",
```

```
                n,aGet/1e6,aIter/1e6,lIter/1e6,dIter/1e6);
        }
    }
}
```

For a fair insertion benchmark, create a fresh collection for each measured trial and use the same insertion position. Avoid repeatedly inserting into a single list without resetting it, because the list size and workload would change during measurement.

Q3 ENGINEER THE HASHCODE–EQUALS CONTRACT

3.1 Contract and Predicted Tests

The Java contract requires that equal objects have the same hash code. If equals() returns true for two Student objects but hashCode() returns different values, HashSet may place them in different buckets and fail to identify the duplicate.

Test	Input	Prediction with broken hashCode	Corrected result
1 — Appears correct	Insert the same object reference twice	Set size 1 because the same reference is recognized	1
2 — Exposes defect	Insert two different Student objects with same regNo	May become size 2 because hashes may differ	1
3 — Mutable key experiment	Insert object, then mutate regNo used by equality/hash	Lookup/removal can fail because object remains in old bucket	Use immutable identity fields; do not mutate key

3.2 Correct Student Class and Complete-the-Code

```
import java.util.*;

class Student {
    private final String regNo;
    private final String name;
    Student(String regNo,String name) {
        this.regNo=Objects.requireNonNull(regNo);
        this.name=name;
    }
    @Override public boolean equals(Object o) {
        if(this==o) return true;
        if(!(o instanceof Student)) return false;
        Student other=(Student)o;
        return regNo.equals(other.regNo);
    }
    @Override public int hashCode() { return Objects.hash(regNo); }
    @Override public String toString() { return regNo+" "+name; }
}
public class Attendance {
    public static void main(String[] args) {
        Set<Student> present=new HashSet<>();
        present.add(new Student("192311001","Arun"));
        present.add(new Student("192311002","Divya"));
        present.add(new Student("192311001","Arun"));
        System.out.println("Unique students: "+present.size());
    }
}
```

Expected corrected output: Unique students: 2. The code uses regNo in both equals() and hashCode(), and regNo is final so the equality key cannot change after insertion.

3.3 How HashSet Works

HashSet uses the element's hashCode() to determine a bucket. It then uses equals() to compare candidates within the bucket. A matching hash code is necessary for equal objects to be found in the same bucket; equals() determines whether they represent the same logical student. Hash collisions between unequal objects are permitted.

If a field used in hashCode() changes after insertion, the object remains stored according to its old bucket placement. A later contains() or remove() computes the new hash and may search a different bucket. This is why fields used for equality should be immutable while an object is stored in a hash-based collection.

3.4 Alternative Structures and Complexity

Structure	Add / de-duplicate	Search	Space
HashSet<Student>	Average O(1)	Average O(1)	O(n)
TreeSet<Student> by regNo	O(log n)	O(log n)	O(n)
HashMap<String,Student>	Average O(1) putIfAbsent	Average O(1) get/containsKey	O(n)

A TreeSet comparator must return zero exactly when the two students should be considered duplicates. A HashMap keyed by regNo makes the unique key explicit and can use putIfAbsent(regNo, student).

Q4 CHOOSING A MAP FOR A PHARMACY INVENTORY

4.1 Map Comparison

Map	put/get	Iteration order	Natural report fit
HashMap	Average O(1)	No guaranteed order	Fast code lookup; sort entries for reports
TreeMap	O(log n)	Sorted by key	Naturally produces medicine-code order
LinkedHashMap	Average O(1)	Insertion order	Naturally produces order-added report

Use TreeMap when the code-sorted report is frequent and should be produced directly. Use LinkedHashMap when insertion-order reporting is important. Use HashMap for high-frequency lookup when report ordering can be prepared separately.

4.2 Implementation

```
import java.util.*;

class Medicine {
    final String code, name;
    int quantity;
    Medicine(String c,String n,int q){code=c;name=n;quantity=q;}
    public String toString(){return code+" "+name+" qty="+quantity;}
}
public class PharmacyMaps {
    public static void main(String[] args) {
        Map<String,Medicine> hash=new HashMap<>();
        Map<String,Medicine> sorted=new TreeMap<>();
        Map<String,Medicine> insertion=new LinkedHashMap<>();
        String[][] data={{"M30","Paracetamol","100"},
                         {"M10","Amoxicillin","60"},
                         {"M20","Cetirizine","80"}};
        for(String[] x:data){
            Medicine m=new Medicine(x[0],x[1],Integer.parseInt(x[2]));
            hash.put(m.code,m); sorted.put(m.code,m); insertion.put(m.code,m);
        }
        System.out.println("Lookup M20: "+hash.get("M20"));
        System.out.println("Code order: "+sorted.values());
        System.out.println("Insertion order: "+insertion.values());
    }
}
```

4.3 Measurement Plan and Acceptability

Use at least three inventory sizes, for example 1,000, 10,000 and 100,000 medicines. Measure 100,000 successful and unsuccessful lookups separately, and measure the time to produce code-order and insertion-order reports. Repeat at least five times after warm-up and report the average.

A practical target for this assignment can be defined as an average in-memory lookup below 1 millisecond for the tested data sizes on the test machine. This is an engineering acceptance target, not a universal guarantee. The remaining cost includes hashing/comparison, memory access, object allocation during loading, and report iteration or sorting.

Hashtable is generally not preferred in new code because its legacy API synchronizes individual methods, does not permit null keys or values, and modern concurrent use cases have clearer alternatives such as ConcurrentHashMap.

4.4 Sorted Array and Binary Search Alternative

If medicines are stored in an array sorted by code, Arrays.binarySearch() provides O(log n) lookup. Insertion into the sorted array costs O(n) because elements must shift. Producing the code-order report is O(n) because the array is already sorted. The array uses O(n) references and has good locality, but resizing and insertion are more expensive than map updates.

Structure	Lookup	Insert/update	Code-order report	Space
Sorted array + binarySearch	O(log n)	O(n) shift	O(n)	O(n)
HashMap	Average O(1)	Average O(1)	O(n log n) if sorted first	O(n)
TreeMap	O(log n)	O(log n)	O(n) iteration	O(n)

Q5 DERIVE A GENERIC RANKING COMPONENT

5.1 Design Choice

A Comparator is preferred for this reusable component because the same Team class may need different ranking policies, and the Leaderboard can also rank players, colleges or quiz teams. Comparable is appropriate when a type has one natural, universal ordering. A Comparator keeps ranking policy outside the data class.

The required order is points descending, goal difference descending, then team name ascending. Use comparingInt(...).reversed() for descending numeric keys and thenComparing for the next key. Avoid subtraction in comparators because integer overflow can produce an incorrect sign.

5.2 Generic Leaderboard and Completed Code

```
import java.util.*;

class Team {
    final String name; final int points; final int goalDiff;
    Team(String n,int p,int g){name=n;points=p;goalDiff=g;}
    public String toString(){return name+"("+points+","+goalDiff+")";}
}
class Leaderboard<T> {
    private final List<T> items=new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order){this.order=order;}
    void add(T item){items.add(item);}
    List<T> ranking(){
        List<T> copy=new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }
    void addAll(List<? extends T> values){items.addAll(values);}
}
public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank =
            Comparator.comparingInt((Team t)->t.points).reversed()
            .thenComparing(Comparator.comparingInt((Team t)->t.goalDiff).reversed())
            .thenComparing(t->t.name);
        Leaderboard<Team> board=new Leaderboard<>(byRank);
        board.add(new Team("Panthers",9,4));
        board.add(new Team("Eagles",9,6));
        board.add(new Team("Cobras",6,1));
        board.add(new Team("Bisons",9,4));
        System.out.println(board.ranking());
    }
}
```

Expected output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. Bisons is above Panthers because their points and goal difference are equal, so the final alphabetical comparison places “Bisons” before “Panthers”.

5.3 Why the Comparator Is Consistent

The comparator performs lexicographic comparison over three ordered keys. First, points determine order. Only when points are equal does goal difference decide. Only when both numeric fields are equal does name decide. Each component comparator is transitive, and lexicographic composition preserves transitivity. If all three keys are equal, compare returns zero.

The method ranking() creates a copy and sorts that copy, leaving the original insertion order unchanged. List<? extends T> is useful in addAll because a list of a subtype of T can safely provide T values to the leaderboard.

5.4 Deliberately Faulty Comparator Test

```
// Faulty: subtraction can overflow and may violate ordering.
Comparator<Team> faulty = (a,b) -> b.points - a.points;
// Better: safe comparison without subtraction.
Comparator<Team> safe = Comparator.comparingInt((Team t)->t.points).reversed();
```

5.5 Comparable and Stable Multi-Sort Alternative

If Team implements Comparable<Team>, compareTo() can encode the one natural ranking. This is concise when every use of Team needs the same order, but less flexible when multiple ranking policies are required. Stable sorting can sort by name, then goal difference descending, then points descending; stability preserves earlier tie-break ordering. However, it performs three O(n log n) sorts rather than one O(n log n) comparator sort. A single comparator chain is easier to reuse for teams, players and colleges.

Approach	Time	Extra space / reuse
One Comparator chain	O(n log n)	Sorting copy O(n); reusable across types with appropriate comparator
Comparable	O(n log n)	O(n) copy if preserving original; natural order belongs to class
Three stable sorts	3 × O(n log n) = O(n log n) asymptotically	Repeated sorting work; tie-break order must be applied in reverse priority sequence

Q6 PRIORITY, ORDER AND FAIRNESS IN A QUEUE

6.1 Design

A plain FIFO Queue serves the earliest arrival regardless of severity, so it cannot guarantee that the most severe patient is treated next. A PriorityQueue is appropriate when a Comparator orders lower severity numbers first. An arrival counter provides FIFO order among equal severities.

6.2 Completed Triage Program

```
import java.util.*;

class Patient {
    final String name; final int severity; final long arrival;
    Patient(String n,int s,long a){name=n;severity=s;arrival=a;}
    public String toString(){return name+"(S"+severity+")";}
}
public class Triage {
    private long counter=0;
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(Comparator.comparingInt((Patient p)->p.severity)
                                      .thenComparingLong(p->p.arrival));
    void admit(String name,int severity) {
        if(severity<1 || severity>5) throw new IllegalArgumentException("Severity must be 1..5");
        queue.add(new Patient(name,severity,counter++));
    }
    Patient next(){return queue.poll();}
    public static void main(String[] args) {
        Triage er=new Triage();
        er.admit("Ravi",3); er.admit("Meena",1); er.admit("John",3);
        er.admit("Asha",5); er.admit("Kumar",1); er.admit("Lata",2);
        StringBuilder order=new StringBuilder();
        Patient p;
        while((p=er.next())!=null) order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Expected output: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5). Meena and Kumar have equal severity, so their increasing arrival counters preserve arrival order. Ravi and John are treated in the same way.

6.3 Hand-Verified Eight-Patient Test

Patient	Severity	Arrival counter	Treatment order
Meena	1	1	1
Kumar	1	4	2
Lata	2	5	3
Ravi	3	0	4
John	3	2	5
Asha	5	3	6
Nila	4	6	7
Dev	2	7	8

Treatment order is severity ascending, then arrival counter ascending. Lata (severity 2, counter 5) precedes Dev (severity 2, counter 7), and severity 4 Nila is treated before severity 5 Asha.

6.4 PriorityQueue Behaviour and Complexity

PriorityQueue.add() and poll() are O(log n), while peek() is O(1). Printing the PriorityQueue directly does not guarantee sorted treatment order because its internal array only maintains the heap property: the head is minimal, but the remaining elements are not fully sorted. To display treatment order, poll from a copy or repeatedly remove from the actual queue.

Operation	PriorityQueue	Sorted LinkedList	Five FIFO queues
Admit	O(log n)	O(n) search/insertion	O(1) append to severity queue
Serve next	O(log n)	O(1) remove first	O(1) check up to five queues
Space	O(n)	O(n) nodes	O(n) patient references + five queue structures

6.5 Starvation and Fairness

A severity-5 patient can wait indefinitely if new severity-1 to severity-4 patients keep arriving. A fair policy can increase effective priority as waiting time grows. For example, after every 30 minutes waiting, reduce the effective severity number by one until it reaches 1. The system should periodically recompute priorities or use scheduled promotion events; changing comparator fields while an object remains inside a heap without re-heapifying is unsafe.

For 5,000 patients per day, the average arrival rate is about 0.058 patients per second over 24 hours. A PriorityQueue with O(log n) admission and removal is suitable computationally. Clinical deployment also requires persistence, concurrency control, audit logs, privacy, downtime handling and clinically approved triage rules.

Q7 — DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE

Selected limitation: repeated String concatenation in Q1

A. Diagnose the Bottleneck

The original ticket builder repeatedly executes s = s + field inside a loop. Because String is immutable, each iteration creates a new String and copies the previous prefix plus the next field. As the number of fields grows, more characters are recopied. This is unnecessary repeated work and the dominant scaling limitation for long tickets or high ticket volume.

B. Define the Design Objective

Objective: preserve exactly the same ticket text while reducing construction from $O(n^2)$ cumulative character copying to O(L) work, where L is the final ticket length. The redesigned builder should create one mutable buffer per ticket and should not share it across threads.

C. Revised Solution

Replace repeated concatenation with StringBuilder. StringBuilder maintains a growable mutable buffer; append adds each field to the end. The final toString() creates the immutable ticket text once. The per-ticket builder is local to the request, so separate booking threads do not contend for the same mutable object.

```
String buildTicket(String[] fields) {
    StringBuilder out = new StringBuilder();
    for (String field : fields) {
        if (out.length() > 0) out.append(System.lineSeparator());
        out.append(field);
    }
    return out.toString();
}
```

Class / flow representation: BookingRequest → TicketBuilder.build(fields) → StringBuilder buffer → final immutable String ticket. The builder is not stored globally and is not shared between threads.

D. Predict Before Implementing

Prediction: the output text remains identical. For short tickets, the measured difference may be small. For large workloads, StringBuilder should reduce average construction time and intermediate String allocation. Expected complexity changes from $O(n^2)$ character copying to O(L), with O(L) buffer space.

E. Implement and Validate

```
import java.util.*;

public class TicketRedesign {
    static String oldBuild(String[] fields) {
        String s="";
        for(String f:fields) s += f + "\n";
        return s;
    }
    static String newBuild(String[] fields) {
        StringBuilder b=new StringBuilder();
        for(String f:fields) b.append(f).append('\n');
        return b.toString();
    }
    static long measure(boolean redesigned,String[] fields,int repetitions) {
        long start=System.nanoTime();
```

```
        long checksum=0;
        for(int i=0;i<repetitions;i++) {
            String result=redesigned?newBuild(fields):oldBuild(fields);
            checksum += result.length();
        }
        if(checksum==0) System.out.println("unused");
        return System.nanoTime()-start;
    }
    public static void main(String[] args) {
        int[] sizes={10,50,500};
        for(int n:sizes) {
            String[] fields=new String[n];
            for(int i=0;i<n;i++) fields[i]="Field-"+i+"-value";
            String oldText=oldBuild(fields), newText=newBuild(fields);
            System.out.println("n="+n+" identical="+oldText.equals(newText));
            int reps=Math.max(100,100000/n);
            measure(false,fields,10); measure(true,fields,10); // warm-up
            long oldNs=measure(false,fields,reps);
            long newNs=measure(true,fields,reps);
            System.out.printf("old %.3f ms; new %.3f ms; speedup %.2fx%n",
                oldNs/1e6,newNs/1e6,(double)oldNs/Math.max(1,newNs));
        }
    }
}
```

Validation procedure: run the same field arrays and repetition counts for both implementations, warm up the JVM, repeat the measurement at least five times, and compare averages. Check that oldText.equals(newText) is true for every size. Record raw run times and environment details (Java version, CPU, operating system). Do not rely on one timing run.

F. Defend the Final Design

The redesign addresses the identified repeated-copying limitation and preserves output correctness. The expected improvement is largest for long tickets and high ticket volume. StringBuilder uses a mutable buffer and may occasionally resize/copy its capacity, but its amortized append cost is linear in total output size. The original String approach remains acceptable for a few fixed, short concatenations where clarity matters more than performance.

Measure	Original: String + loop	Redesigned: StringBuilder
Correct output	Must match	Must match exactly
Character-copying complexity	$O(n^2)$ for n growing prefixes	O(L) total output length
Intermediate objects	Many intermediate String objects	One builder buffer and final String
Thread safety	String values immutable; safe to share	Builder itself is not thread-safe; use one per request/thread
Best use	Small fixed concatenations	Repeated/large dynamic text construction

FINAL CONCLUSION

The engineering choices follow the workload: StringBuilder for repeated per-ticket construction; ArrayList for indexed playlists and ArrayDeque for double-ended queues; immutable Student identity with a consistent equals/hashCode pair; TreeMap or LinkedHashMap when report order is required; Comparator chains for reusable multi-key ranking; and PriorityQueue with an arrival sequence for severity-first stable triage. The Day-4 redesign replaces repeated immutable String concatenation with a per-request StringBuilder and validates that output remains identical.